Reviewer Pack — Minimal p-Adic Valuation Width and Resolution Proof Width Co…

Entry 5bb6845bd157 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 04:20:15 UTC

Minimal p-Adic Valuation Width and Resolution Proof Width Correlation

Entry ID: `5bb6845bd157` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-31 04:20:15 UTC

1. Conjecture

Field A (mathematical branch): p-Adic Analysis **Field B** (complexity object): Resolution Proof Complexity

Statement:

> For every Tseitin formula π with n variables, the minimal p-adic valuation width (MVW(π)) of its associated Boolean function is linearly correlated with its resolution proof width w(π), such that MVW(π) = Θ(w(π)).

Rationale (proposer's reasoning):

> p-Adic analysis provides a framework to study the arithmetic structure of numbers, which may reveal hidden relationships in computational complexity, specifically in the width of resolution proofs. The p-adic valuation width could act as a quantitative measure for the difficulty of resolving logical statements, potentially leading to new lower bounds.

Taxonomy category: `p-adic_analysis_resolution_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9c877cf0525712fc`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all Tseitin formulas π with n variables (n ≤ 40), the correlation coefficient between MVW(π) and w(π) across 30 random seeds is greater than or equal to 0.7, without any seed producing a correlation coefficient less than 0.5.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: `ADJACENT_OK` against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `p-Adic Analysis AND Resolution Proof Complexity` - `Tseitin formula AND p-adic valuation width AND resolution proof width` - `correlation p-adic valuation width resolution proof complexity`

Top relevant hits considered: - [http://arxiv.org/abs/math-ph/0512018v2] On Phase Transitions for P-Adic Potts Model with Competing Interactions on a Cayley Tree - [http://arxiv.org/abs/2408.00810v3] p-adic Equiangular Lines and p-adic van Lint-Seidel Relative Bound - [http://arxiv.org/abs/2201.08874v2] Note on p-adic Local Functional Equation - [http://arxiv.org/abs/1510.02114v3] The p-adic Gross-Zagier formula on Shimura curves - [http://arxiv.org/abs/cond-mat/0005533v2] Evolution of interfaces and expansion in width - [http://arxiv.org/abs/2103.06864v4] On generalized Iwasawa main conjectures and p-adic Stark conjectures for Artin motives - [http://arxiv.org/abs/1211.0398v1] Extending valuations to formal completions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        literals = [f'x{i}' for i in range(1, n+1)]
        clauses = []

        # Generate clauses for each literal
        for lit in literals:
            clauses.append([lit])

        # Generate clauses for implications
        for i in range(1, n+1):
            for j in range(i+1, n+1):
                clauses.append([f'-x{i}', f'x{j}'])
                clauses.append([f'-x{j}', f'x{i}'])

        # Generate the final clause
        final_clause = [f'-x{i}' for i in range(1, n+1)]
        clauses.append(final_clause)
```

```
    return literals, clauses

def evaluate_formula(clause, assignment):
    return any(lit[0] == '-' and not assignment[int(lit[1:]) - 1] or lit[0] != '-' and assignm

def dpll(clauses, assignment):
    if not clauses:
        return True
    unit_clauses = [c for c in clauses if len(c) == 1]
    if unit_clauses:
        literal = unit_clauses[0][0]
        new_assignment = assignment[:]
        new_assignment[int(literal[1:]) - 1] = literal[0] == '-'
        return dpll([c for c in clauses if not evaluate_formula(c, new_assignment)], new_assig

    literal = random.choice(clauses[0])
    new_assignment = assignment[:]
    new_assignment[int(literal[1:]) - 1] = literal[0] == '-'
    if dpll([c for c in clauses if not evaluate_formula(c, new_assignment)], new_assignment):
        return True
    new_assignment[int(literal[1:]) - 1] = literal[0] != '-'
    return dpll([c for c in clauses if not evaluate_formula(c, new_assignment)], new_assignmen

def compute_p_adic_valuation_width(formula):
    n = len(formula)
    values = []

    for i in range(2**n):
        assignment = [bool(i & (1 << j)) for j in range(n)]
        value = evaluate_formula(formula, assignment)
        if value:
            p_adic_val = 0
            while i % 2 == 0:
                i //= 2
                p_adic_val += 1
            values.append(p_adic_val)

    return min(values) if values else 0

def compute_resolution_proof_width(formula):
    literals, clauses = formula
    n = len(literals)

    def resolve(clause1, clause2):
        new_clauses = []
        for lit1 in clause1:
            for lit2 in clause2:
                if abs(int(lit1)) == abs(int(lit2)):
                    continue
                new_lit = f'-{int(lit1)}' if int(lit1) * int(lit2) < 0 else str(abs(int(lit1))
                new_clause = [l for l in clause1 + clause2 if l != lit1 and l != lit2]
                new_clause.append(new_lit)
                new_clauses.append(new_clause)
        return new_clauses

    def dpll(clauses, assignment):
```

```
            if not clauses:
                return True
            unit_clauses = [c for c in clauses if len(c) == 1]
            if unit_clauses:
                literal = unit_clauses[0][0]
                new_assignment = assignment[:]
                new_assignment[int(literal[1:]) - 1] = literal[0] == '-'
                return dpll([c for c in clauses if not evaluate_formula(c, new_assignment)], new_a

            literal = random.choice(clauses[0])
            new_assignment = assignment[:]
            new_assignment[int(literal[1:]) - 1] = literal[0] == '-'
            if dpll([c for c in clauses if not evaluate_formula(c, new_assignment)], new_assignmen
                return True
            new_assignment[int(literal[1:]) - 1] = literal[0] != '-'
            return dpll([c for c in clauses if not evaluate_formula(c, new_assignment)], new_assig

        proof_width = 0
        while clauses:
            clause = random.choice(clauses)
            clauses.remove(clause)
            new_clauses = []
            for other_clause in clauses:
                if any(lit in clause and lit[0] == '-' != other_lit[0] for lit, other_lit in zip(c
                    new_clauses.extend(resolve(clause, other_clause))
            clauses.extend(new_clauses)
            proof_width += 1

        return proof_width

    n = random.choice([5, 10, 15, 20, 30, 40])
    formula = generate_tseitin_formula(n)

    mvw = compute_p_adic_valuation_width(formula)
    wp = compute_resolution_proof_width(formula)

    return {
        "metric_name": "correlation",
        "metric_value": Fraction(mvw * wp).limit_denominator(),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": mvw * wp > 0,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 31)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean = sum(results) / len(results)
```

```
    std = math.sqrt(sum((x - mean)**2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r >= 0.7) / len(results)

    if all(r >= 0.7 for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(r < 0.5 for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result < 0.5)
        print(f"RESULT: FALSIFIED counterexample='correlation_coefficient_less_than_0.5' first_fai
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(seeds)}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_642441f2.py", line 146, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_642441f2.py", line 128, in run_trial
    mvw = compute_p_adic_valuation_width(formula)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_642441f2.py", line 68, in compute_p_adi
    value = evaluate_formula(formula, assignment)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_642441f2.py", line 42, in evaluate_form
      return any(lit[0] == '-' and not assignment[int(lit[1:]) - 1] or lit[0] != '-
' and assignment[int(lit) - 1] for lit in clause)
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_642441f2.py", line 42, in <genexpr>
      return any(lit[0] == '-' and not assignment[int(lit[1:]) - 1] or lit[0] != '-
' and assignment[int(lit) - 1] for lit in clause)
                                                                                ^^^^^^^^
TypeError: int() argument must be a string, a bytes-like object or a real number, not 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed during execution, which prevented the production of data necessary to evaluate the conjecture. | next: Review and debug the test code to ensure it can run to completion without errors.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21511
2	preregistration	ollama_remote	glm4:latest	0	0	9375
3	novelty	ollama_remote	glm4:latest	0	0	8276
4	novelty	ollama_remote	glm4:latest	0	0	9876
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21881
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18505
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16089
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17540
9	judge	ollama_remote	glm4:latest	0	0	13788

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 136841 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/5bb6845bd157.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5bb6845bd157.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/5bb6845bd157.tar.gz` (if generated)